

课程笔记

Superpowers Skills: Vibe Coding 时代的 软件开发圣经

五道口纳什 & AI

2026 年 3 月 28 日

Superpowers Vibe Coding 圣经

视频作者/频道: 五道口纳什

发布日期: 2025

视频时长: 约 40 分钟

视频链接: 未填写

目录

1 引言：Vibe Coding 时代的质量困境	3
2 Skills 的加载机制	3
2.1 Codex 中的 System Prompt 架构	3
2.2 Skill 的匹配与加载流程	3
2.3 安装与更新	4
2.4 本章小结	4
3 总控 Skill: Using Superpowers	4
3.1 总控流程	4
3.2 流程的形式化定义	4
3.3 本章小结	5
4 Brainstorming (头脑风暴)	5
4.1 目标与原则	5
4.2 流程步骤	5
4.3 本章小结	5
5 Writing Plans (实施计划)	5
5.1 目标与原则	5
5.2 产出物：Plan.md	6
5.3 计划完成后的选择	6
5.4 本章小结	6
6 TDD: 红绿重构	6
6.1 三阶段纪律	6
6.2 为什么必须“亲眼看着测试失败”?	7
6.3 验证前必须有证据	7
6.4 本章小结	7
7 执行模式	7
7.1 Sequential Development (渐进式开发)	7
7.2 Parallel Agents (并发 Subagent)	7
7.3 Worktree 隔离	7
7.4 本章小结	7
8 Systematic Debugging (系统化调试)	7
8.1 四阶段流程	7
8.2 完成前验证 Skill	8
8.3 本章小结	8

9 Code Review 体系	8
9.1 Review 的时机与原则	8
9.2 请求 Code Review 的 Checklist	8
9.3 接收 Code Review 的准则	9
9.4 SOLID 与关注点分离	9
9.5 本章小结	9
10 代码复用与 DRY	10
10.1 本章小结	10
11 如何编写好的 Skills	10
11.1 本章小结	10
12 总结与延伸	10
12.1 拓展阅读	11

1 引言：Vibe Coding 时代的质量困境

在 AI 辅助编程 (Vibe Coding) 快速普及的今天, 开发者面临一个核心问题: **如何在使用 Codex、Claude Code 等纯自然语言编程工具时, 依然保证代码的准确性与可维护性**。本期围绕开源项目 **Superpowers** (GitHub Star 7 万 +) 这一组高质量 Skills, 系统性地回答这一问题。

Vibe Coding 的工具光谱

讲者将当前 AI 编程工具分为三个层级:

1. **VS Code (无 AI 插件)**: 纯粹的代码编辑器 / IDE。
2. **Cursor**: 带有 Tab 模型 (大规模 on-policy RL 训练) 的轻量级 AI 辅助, 支持对话式开发与 diff GUI。
3. **Codex / Claude Code**: 纯自然语言编程 CLI, 无 IDE, 只需描述需求并验证输出。

传统方法中, 代码质量通过 **TDD** (测试驱动开发)、**Code Review**、**Pre-commit Hook** (如 Ruff 静态检查) 等流程来保障。在 Vibe Coding 时代, Superpowers 将这些顶级软件工程实践”硬编码”为 AI Coding Agent 的规范操作系统——通过 Skill 指令而非 Hook 脚本, 强制 Agent 克服大模型常见的幻觉、懒惰和讨好倾向。

核心理念: ”慢其实快”

不加任何 Skill、不加任何约束, 模型可能很快就产出代码, 但代码不准、不可维护。有了 Superpowers 的约束后, 虽然每一步都更慎重, 但代码质量有保障, 长期迭代效率更高——这就是”慢其实快”。

2 Skills 的加载机制

2.1 Codex 中的 System Prompt 架构

Codex 在用户发出第一条请求之前, 已经植入了四条系统消息:

1. **System 级提示**: 系统级约束。
2. **Developer 消息**: 权限与沙箱规则。
3. **User 消息 (注入)**: 包含 `agents.md` 和所有可用 Skills 的描述——这是 Skills 的**第一次暴露**。
4. **Developer 消息**: 定义协作模式 (Plan mode / Agent mode)。

2.2 Skill 的匹配与加载流程

1. 用户发出真实请求。
2. Coding Agent 基于 query **匹配最合适的 Skill**。
3. Agent 调用 `execute_commands` 工具, 执行 `cat` 命令加载对应 `skill.md` 的全文。
4. Skill 内容进入上下文, 后续所有交互均基于该 Skill 的描述执行。

5. 一个 turn 结束后，用户可继续迭代需求。

agents.md 的两级来源

- **系统级**：`~/.codex/agents.md`（全局配置）。
- **项目级**：代码仓库根目录下的 `agents.md`（项目专属指令）。

两者都会被注入到第三条 User 消息中。

2.3 安装与更新

- Claude Code 通过 plugin 机制安装 Superpowers。
- Codex 通过自然语言让 Agent 自行安装。
- 更新方式：进入 Superpowers 目录执行 `git pull`，下次开启新会话即自动加载。
- 当前版本支持**自主发现**：输入 `/skills` 可列出所有可用 Skill。

2.4 本章小结

Skills 通过 Prompt Injection 的方式，在 Codex 的第三条 User 消息中暴露目录；Agent 根据用户 query 动态匹配并用 `cat` 命令加载具体的 `skill.md`，从而将软件工程流程注入模型上下文。

3 总控 Skill: Using Superpowers

3.1 总控流程

Using Superpowers 是第一个被触发的 Skill（“use when starting any conversation”）。它定义了整体开发链路：

1. **Brainstorming**（头脑风暴）：与用户多轮对话确认修改边界。
2. **Writing Plans**（写实施计划）：将设计转化为可执行步骤。
3. **Development**（开发）：以 TDD 方式先写测试再写实现。
4. **Verification**（验证）：完成前真实运行命令检查结果。

Skill.md 中的强制触发指令

Superpowers 的 `skill.md` 中包含用 XML 标签包裹的强制指令：“极度重要——如果你认为有 1% 的机会可以使用某个 Skill，一定要触发它。”

3.2 流程的形式化定义

Skill 内部使用 **Dot 语言**（Graphviz）定义流程图——用形式化代码语言描述 SOP，使语言模型更易消化理解。流程中还要求 Agent **宣称**（declare）自己正在使用什么 Skill、出于什么目的，并定义一组 **Checklist** 检查是否完成要求。

3.3 本章小结

Using Superpowers 是 Agent 的”操作系统入口”，通过 Dot 语言定义的有向图来约束 Agent 行为，并通过宣称机制和 Checklist 确保每一步都有迹可查。

4 Brainstorming（头脑风暴）

4.1 目标与原则

Brainstorming 解决的是 **What & Why**——”我们要建什么、为什么这样建”。

- 一次只问一个确认性问题。
- 理解用户的真实意图（Purpose）、约束和成功准则。
- 先探索上下文，提出 **2-3 个方案**，分析各自的 Trade-off。
- 用户确认后，形成并保存 **Design Doc** (docs/plans/<date>-<topic>-design.md)。

头脑风暴阶段的禁令

在 Brainstorming 阶段，**禁止调用任何实现工具、禁止写任何代码**。此阶段的产出物纯粹是概念与决策文档，侧重宏观架构、数据流向、组件交互、异常处理。

4.2 流程步骤

1. 探索项目上下文。
2. 向用户提出确认性问题，明确问题边界。
3. 提出 2-3 个解决方案，分析 Trade-off。
4. 呈现 Design Section，等待用户批准。
5. 用户批准后，自动调用 Writing Plans Skill。

4.3 本章小结

头脑风暴是需求澄清的必经阶段，产出 Design Doc，为后续实施计划提供架构基础。其核心纪律是”只设计不编码”。

5 Writing Plans（实施计划）

5.1 目标与原则

Writing Plans 解决的是 **How**——”如何实现”。它将宏观设计转化为**零上下文可执行的细粒度**步骤。

三大准则

1. **DRY** (Don't Repeat Yourself): 避免写重复代码和重复逻辑。
2. **YAGNI** (You Aren't Gonna Need It): 不做“可能以后用得上”的功能。
3. **TDD**: 先写测试再写实现, 频繁提交、小步迭代。

5.2 产出物: Plan.md

计划文档 (Plan.md) 包含极其具体的细节:

- 要创建哪个确切路径的文件。
- 要修改哪几行。
- 要写什么样的失败测试。
- 要运行什么确切的命令来验证。

消除上下文依赖

计划的编写标准是: 假设最终写代码的工程师 (或另一个 Subagent) 对项目一无所知, 仅凭此计划即可完成开发。因此可以另开一个会话, 让 Codex / Claude Code / 人类工程师直接执行。

5.3 计划完成后的选择

计划写完后不写代码, 而是询问用户:

- 在当前会话中渐进式执行 (Sequential Development)。
- 开新会话批量执行 (Parallel Agents)。

5.4 本章小结

Writing Plans 将架构设计“翻译”为任何人/Agent 都能执行的微观步骤清单, 是从“想”到“做”的关键桥梁。

6 TDD: 红绿重构

6.1 三阶段纪律

红-绿-重构

1. **红 (Red)**: 先写一个会失败的测试——因为功能尚未实现, 测试必然报错。
2. **绿 (Green)**: 写最少量的代码使测试通过。
3. **重构 (Refactor)**: 在保持绿色的前提下优化代码结构。

6.2 为什么必须“亲眼看着测试失败”？

事后测试证明不了任何事情

- 写完测试后必须运行，并看到**预期的报错**。
- 如果测试直接通过，说明测的东西是错的，或测的是已经存在的行为。
- 先写代码再补测试，往往只会测试“代码现在的行为”，而不是“代码应该的行为”——这就是**技术债务**的来源。

6.3 验证前必须有证据

在宣称完成或测试通过前，必须**真实运行命令/工具**并检查输出——大量使用断言（assert），对结果有明确预期。

6.4 本章小结

TDD 的核心纪律是“先红后绿再重构”，强制 Agent 在每一步都用真实的测试结果作为证据，杜绝“假装完成”。

7 执行模式

7.1 Sequential Development（渐进式开发）

在当前会话中按计划逐步执行：每完成一个 task，实施 → 验证 → Code Review，再进入下一个。适用于需要人类步步把关的复杂高风险任务。

7.2 Parallel Agents（并发 Subagent）

当计划中的任务彼此独立时，为每个任务派发新的 Subagent 并发执行。完成后进行双重机器审查（规范 + 质量）。

7.3 Worktree 隔离

使用 Git Worktree 隔离当前分支，在单个计划执行中强调“实现 → 规范 → Review 代码质量”的顺序。

7.4 本章小结

Superpowers 提供了从单线程渐进到多 Agent 并发的多种执行模式，灵活适配不同复杂度的开发场景。

8 Systematic Debugging（系统化调试）

8.1 四阶段流程

遇到 Bug 或测试失败时触发，**禁止盲目猜测或试错式修复**：

1. **根因调查** (Root Cause Analysis): 找到真正的原因。
2. **模式分析** (Pattern Analysis): 识别重复出现的问题模式。
3. **提出假设** (Hypothesis): 基于证据提出修复方案。
4. **最小化实现与验证** (Minimal Fix & Verify)。

三次循环未解决则架构有问题

如果上述四阶段循环了三次仍无法解决, 说明可能存在架构层面的问题, 需要回到 Brainstorming 重新审视设计。

8.2 完成前验证 Skill

AI 喜欢”假装完成”或过度自信。此 Skill 要求: 在宣布任务完成、测试通过、准备提交之前, 必须**真实运行验证命令**, 并根据实际输出来宣称完成。

8.3 本章小结

系统化调试用四阶段闭环替代”拍脑袋式”修复, 三次未果则升级为架构问题, 避免在错误方向上越陷越深。

9 Code Review 体系

9.1 Review 的时机与原则

Review Early, Review Often

- 不是在合并主分支前才审查, 而是**每个细粒度子任务完成后都必须强制触发审查**。
- 防止设计错误”污染”下一个任务。
- Code Review 的输入: 用户原始需求/计划 + Git Diff。

9.2 请求 Code Review 的 Checklist

1. **代码质量**: 关注点分离、解耦、错误处理、重复代码、边界 Case 覆盖。
2. **架构**: 设计角色是否合理、可扩展性、性能影响、安全隐患。
3. **测试**: 是否测了真实逻辑 (而非只是 Mock)、边界覆盖率、集成测试。
4. **需求**: 是否满足计划的所有要求、无功能蔓延、达到生产级完成度。

行为红线

- 严禁不看代码就说”看起来不错”。
- 严禁将次要问题标为致命错误。
- 严禁含糊其词——必须给出最终结论。

9.3 接收 Code Review 的准则

Code Review 的本质是基于客观代码的技术评估，而非社交表现：

- **追求技术严谨性**：决策基于事实、测试结果、系统架构现状，而非直觉或权威。
- **拒绝表演性同意**：不在未验证技术可行性的前提下盲目同意 Reviewer 建议。
- **YAGNI 原则**：若 Reviewer 建议添加当前系统根本没有调用的功能，应拒绝。
- **验证先于执行**：收到反馈后第一步是去代码库中验证，而非立即修改。
- **有理有据的技术反驳 (Push Back)**：发现 Reviewer 建议有误时，使用技术推理反驳——反驳不是为了争吵，而是为了保护代码库。

消除模糊性

如果 Reviewer 提了 6 点意见，其中 2 点描述不清，**严禁先做懂的 4 点、把不懂的 2 点留到最后**。必须立刻停下来，先询问清楚所有细节——因为软件模块之间高度耦合，部分理解会导致错误的实现方向，返工成本极高。

行动胜于雄辩

当 Reviewer 意见正确时，严禁使用感激或赞美的废话。正确做法是**直接修复并简短陈述事实**。高效工程文化中，代码本身就是最好的回应。

9.4 SOLID 与关注点分离

Code Review 中涉及的架构设计模式：

1. **单一职责原则 (SRP)**：避免一个类/文件承担所有事情，必须解耦。
2. **开闭原则 (OCP)**：对扩展开放、对修改关闭。
3. **里氏替换原则 (LSP)**：子类可替换父类。
4. **接口隔离原则 (ISP)**：接口应细粒度。
5. **依赖倒置原则 (DIP)**：依赖抽象而非具体实现。

关注点分离是宏观架构哲学：将软件划分为具有不同职责的独立模块，每个模块只解决一个特定的关注点。

9.5 本章小结

Code Review 体系包含”请求审查”和”接收审查”两个 Skill，通过 Checklist 和行为红线确保审查的技术严谨性，同时用 SOLID 原则和关注点分离指导架构评估。

10 代码复用与 DRY

在 Vibe Coding 场景下，模型受限于 Context Window，容易只看部分代码就开始修改，导致工具函数（Utils）出现大量冗余和重复代码。

Superpowers 在 Writing Plans 和 Code Review 中反复强调：

- **DRY** (Don't Repeat Yourself)：杜绝重复代码。
- **YAGNI** (You Aren't Gonna Need It)：坚决杜绝过度设计和过度抽象。

此外，Superpowers 还提供了 Writing Skills 这一 Skill，强调将解决问题的过程提炼为**可复用的模式**，而非一次性脚本。

10.1 本章小结

DRY 与 YAGNI 是 Superpowers 贯穿始终的两大原则，前者防止冗余，后者防止过度工程化，两者构成代码质量的动态平衡。

11 如何编写好的 Skills

讲者总结了编写高质量 Skills 的要素：

1. **足够抽象**：Skills 必须是通用的（Common）、有原则的（Principled）、可复用的。
2. **形式化描述**：使用非常 Formal 的、逻辑性的描述语言（如 Dot 语言流程图）。
3. **专家经验沉淀**：一定是资深软件开发人员的实践凝练。
4. **Writing 能力**：需要良好的技术写作能力，但可以借助语言模型辅助润色。

11.1 本章小结

好的 Skill 是”专家经验 + 形式化表达”的结晶，既要有深度的工程理解，又要有让模型易于执行的结构化描述。

12 总结与延伸

本期以 Superpowers 项目为核心，系统讲解了 Vibe Coding 时代如何”驾驭”（Harness）AI Coding Agent，使其写出”又对又好又准”的代码。核心要点回顾：

1. **Skills 加载机制**：通过 Prompt Injection 在 Codex 系统消息中暴露 Skill 目录，Agent 动态匹配并加载。
2. **总控流程**：Using Superpowers → Brainstorming → Writing Plans → TDD Development → Verification。
3. **TDD 纪律**：红-绿-重构三阶段，”亲眼看着测试失败”是不可跳过的第一步。
4. **系统化调试**：四阶段闭环，三次未果则升级为架构问题。

5. **Code Review 双向体系**: 请求审查 (Checklist 驱动) + 接收审查 (技术严谨性 + Push Back)。
6. **DRY & YAGNI**: 贯穿始终的两大原则。

Superpowers 的意义不仅仅是一组 Prompt，而是将一套经过验证的顶级软件工程实践”硬编码”为 AI 时代的规范操作系统。它代表了 **Harness Engineering**——通过 Skill 这根”缰绳”，驾驭越来越强的 Coding Agent，在效率与质量之间找到最优平衡。

12.1 拓展阅读

- Superpowers GitHub 仓库
- Kent Beck, *Test-Driven Development: By Example*
- Robert C. Martin, *Clean Code*
- Martin Fowler, *Refactoring: Improving the Design of Existing Code*